Oxford Cambridge and RSA – style practice paper

GCSE (9–1) Computer Science

J277/02 Computational thinking, algorithms and programming

Topic 2.3: Producing robust programs

Candidate forename		**Candidate surname**	
Centre number		**Candidate number**	

Practice questions in the style of OCR GCSE Computer Science (J277). Every question is original; the layout, command words, mark allocations and mark scheme follow the real exam format.

Time allowed	About 85 minutes (roughly 1 minute per mark)
Instructions	• Use black ink. HB pencil may be used for graphs and diagrams only. • Answer **all** the questions. • Write your answer to each question in the space provided. • Where the question asks you to write code or pseudocode, you may use any suitable high-level language or the OCR Reference Language; marks are for the logic, not exact syntax. • Do **not** use a calculator or any other electronic device unless told to.
Information	• The total mark for this paper is **79**. • The marks for each question are shown in brackets [ ]. • Quality of extended responses will be assessed in questions marked with an asterisk (*). • Where a question asks you to *state* or *give*, a short answer is enough; *describe* and *explain* need more detail.

Name: _____ Date: _____ Mark: _____ / 79

1 A programmer is designing a program that allows students to book places on school trips. The programmer uses defensive design.

(a) State what is meant by **defensive design**. **[2]**

(b) Describe **two** ways a user might misuse the booking program. **[2]**

(c) Give **two** reasons why a programmer would use defensive design. **[2]**

2 The booking program uses validation and authentication.

(a) State what is meant by **validation**. **[1]**

(b) Draw **one** line from each validation check to the data it checks. **[4]**

Validation check	**Example**
Range check	A name has been entered (the field is not empty)
Length check	A quantity is a whole number (integer)
Presence check	An age is between 11 and 16
Type check	A password has at least 8 characters

(c) The code below should only accept a number between 1 and 10. Describe the **error** in the code and how to correct it. **[2]**

```
num = int(input("Enter a number from 1 to 10"))
while num < 1 AND num > 10
    num = int(input("Invalid. Try again"))
endwhile
```

(d) Explain the difference between **validation** and **verification**. **[2]**

(e) State **two** ways a user could be authenticated when logging in to the booking system. **[2]**

(f) Explain why a system that uses only a four-digit PIN for authentication may be insecure. **[2]**

3 A programmer has written part of a program. A colleague will need to maintain it later.

(a) Give **three** features of the code that make it difficult to maintain. **[3]**

```
a = input("")
b = input("")
c = 0
for x = 1 to 10
c = c + int(a) * int(b)
next x
print(c)
```

(b) Describe **three** ways a programmer can make code easier to maintain. **[3]**

(c) Explain why code should be easy for other programmers to understand. **[2]**

4 A school program should only accept the age of a student in Year 7 to Year 11, between 11 and 16 inclusive. The programmer tests it with different data.

(a) State what is meant by **boundary data**. **[1]**

(b) Complete the test table. The first row has been completed for you. **[5]**

Test data	Type of test data	Expected result
14	Normal	Accepted
11		
10		
25		
"abc"		

(c) Explain why the programmer should test with **erroneous** data. **[2]**

(d) State **two** reasons why test plans are written before a program is tested. **[2]**

5 Errors can be syntax errors or logic errors. The following code has been written to count how many of 5 marks are passes (50 or more).

```
count = 0
for i = 1 to 5
    mark = int(input("Enter mark"))
    if mark > 50 then
        count = count + 1
    endif
next i
print("Number of passes: " + count)
```

(a) State what is meant by a **syntax error**. **[2]**

(b) State what is meant by a **logic error**. **[2]**

(c) Identify the **logic error** in the code and explain what effect it has. **[2]**

(d) The last line causes a run-time error. Suggest why and write the corrected line. **[2]**

(e) State what the user would see if the code was run with the input 50, 50, 20, 70, 49. **[2]**

6 A programmer is developing a large program and tests it as she goes.

(a) Describe the difference between **iterative testing** and **final (terminal) testing**. **[2]**

(b) Give **two** advantages of iterative testing. **[2]**

(c) State why the programmer should still carry out final testing. **[1]**

(d) Describe how a programmer can use a **trace table** to find a logic error. **[2]**

7 Writing robust code.

(a) Write a program that asks the user to enter a whole number between 1 and 100. The program must keep asking until a valid number is entered, and then output "Thank you". **[5]**

(b) A login program allows the user to enter a username and password. Describe how the program could be made more secure against someone trying to guess the password. **[3]**

(c) Explain why comments in a program do not affect how the program runs. **[1]**

8 Extended response questions.

(a) *A company is developing a program that customers use to order food online. Discuss the steps the company should take to make sure the program is secure and robust. **[8]**

(b) *Evaluate the importance of testing during and after the development of a program. Refer to types of test data and types of error in your answer. **[8]**

END OF QUESTIONS

MARK SCHEME – Topic 2.3: Producing robust programs

How to use this mark scheme: mark your own work in a different colour. A bullet (•) in the Answer column is a separate marking point worth one mark unless stated. A slash (/) separates alternative answers that are equally acceptable. Words in brackets are not needed. **Accept** = acceptable; **Ignore** = neither credited nor penalised; **Do not accept** = contradicts or spoils the answer. For programming questions, any valid solution that works is acceptable and syntax errors are not penalised if the intent is clear. The code in the answers uses the OCR Reference Language style.

Question	Answer	Marks	Guidance
1(a)	• designing a program to anticipate how it might be misused / what could go wrong ; • so that it continues to work correctly / securely and prevents errors / attacks ;	2 AO1 2.3.1	
1(b)	• entering incorrect / invalid data (e.g. letters for a phone number) ; • leaving required fields blank ; • trying to guess passwords / book for someone else ; • entering very large / negative numbers ;	2 AO3 2.3.1	Any two sensible misuses.
1(c)	• to make the program more robust / reduces crashes ; • to protect against unauthorised access / malicious use ; • so the program is reliable / more secure / less maintenance ;	2 AO1 2.3.1	Any two.
2(a)	checking that data entered is reasonable / sensible / meets set rules ;	1 AO1 2.3.1	Do not accept 'checking it is correct'.
2(b)	• range check: age is between 11 and 16 ; • length check: a password has at least 8 characters ; • presence check: a name has been entered ; • type check: a quantity is a whole number ;	4 AO1 2.3.1	One mark per correct match.
2(c)	• the check uses AND / the loop condition is wrong: `num < 1 AND num > 10` can never be true ; • should be `num < 1 OR num > 10` ;	2 AO3 2.3.1	
2(d)	• validation checks that data is reasonable / follows rules ; • verification checks that data entered matches the original / intended (e.g. entering twice) ;	2 AO2 2.3.1	
2(e)	• username and password ; • two-factor authentication (code sent to phone) ; • biometrics (fingerprint / face) ; • CAPTCHA / security questions ;	2 AO1 2.3.1	Any two.
2(f)	• only 10 000 possible combinations ; • easy to guess / brute force quickly ;	2 AO3 2.3.1	
3(a)	• no comments explaining what the code does ; • meaningless variable names (a, b, c, x) ; • no indentation / poor layout ; • no use of subprograms / repeated code ;	3 AO3 2.3.1	Any three. Accept: magic numbers (hard-coded values).
3(b)	• add comments ; • use meaningful variable and subprogram names ; • indent code to show the structure ; • use subprograms to split the code into smaller parts ; • use constants / avoid repeated code ;	3 AO1 2.3.1	Any three.

Question	Answer	Marks	Guidance
3(c)	• programs are often updated / fixed by different people ; • so they must understand the code to change it quickly without introducing errors ;	2 AO2 2.3.1	
4(a)	data at the edge of (or just outside) the range the program accepts ;	1 AO1 2.3.2	
4(b)	• 14 normal: accepted ; • 11 / 16: boundary: accepted ; • 10 / 17: boundary: rejected ; • 25 / −3: invalid: rejected ; • "abc": erroneous: rejected (with message) ;	5 AO3 2.3.2	11 (or 16): Boundary / accepted. 10 (or 17): Boundary / rejected. 25: Invalid / rejected. "abc": Erroneous / rejected. Award 1 mark per correct row, 1 for the first row logic.
4(c)	• to check the program does not crash when the wrong type of data is entered ; • so that the program shows an error message and asks again ;	2 AO2 2.3.2	
4(d)	• to make sure every part / all types of data are tested ; • so expected results are known in advance to compare ; • to repeat the tests after changes ;	2 AO1 2.3.2	Any two.
5(a)	• an error in the rules / grammar of the programming language ; • it stops the program from running / being translated ;	2 AO1 2.3.2	
5(b)	• an error in the design / logic of the program ; • the program runs but gives unexpected or incorrect results ;	2 AO1 2.3.2	
5(c)	• mark > 50 should be mark >= 50 ; • a mark of exactly 50 would not be counted as a pass ;	2 AO3 2.3.2	
5(d)	• count is an integer and cannot be joined to a string without casting ; • print("Number of passes: " + str(count)) ;	2 AO3 2.3.2	
5(e)	• Number of passes: 1 ; • (only 70 is counted because of the logic error) ;	2 AO3 2.3.2	Accept error message if candidates identify the run-time error in the last line (after the logic error output is known).
6(a)	• iterative testing is done during development, testing each module / part as it is written ; • final testing is done once the program is complete, testing the whole program ;	2 AO1 2.3.2	
6(b)	• errors are found / fixed early (when they are easier and cheaper to fix) ; • individual parts can be checked before they are combined ; • reduces the number of errors at the final stage ;	2 AO2 2.3.2	Any two.
6(c)	to check the whole program works together as required / with the user's requirements ;	1 AO2 2.3.2	
6(d)	• follow the code line by line, recording the value of each variable after each step ; • compare with expected values to find where the code gives the wrong result ;	2 AO2 2.3.2	
7(a)	• input inside a loop ; • range check 1 to 100 ; • loop repeated while invalid ; • error message shown / new input asked ; • "Thank you" output after valid ;	5 AO3 2.3.1	Example: do n = int(input("Enter 1-100")) until n >= 1 AND n <= 100 print("Thank you")

Question	Answer	Marks	Guidance
7(b)	• limit the number of attempts ; • lock the account / delay after too many failed attempts ; • force strong passwords (length, mixed characters) / two-factor authentication ;	3 AO3 2.3.1	
7(c)	comments are ignored by the translator ;	1 AO2 2.3.1	
8(a)	**Level 3 (high), 6–8 marks**: Thorough knowledge and understanding of the topic with detailed, accurate and relevant points. A balanced, well-structured answer reaching a justified conclusion where asked. Technical terms used correctly throughout. **Level 2 (mid), 3–5 marks**: Reasonable knowledge and understanding. Some relevant points made, but the answer lacks detail, balance or a clear structure in places. **Level 1 (low), 1–2 marks**: Basic knowledge. A few simple, mostly isolated points with little or no development or linking. **Level 0, 0 marks**: No response or no response worthy of credit. **Indicative content** (credit any other valid points): • defensive design: anticipate misuse, such as wrong data, empty fields, SQL injection attempts • input validation: range, length, type, presence and format checks; sanitise input; error messages and re-entry • authentication: strong passwords, limit login attempts, two-factor authentication, CAPTCHA • maintainability: comments, meaningful names, indentation, subprograms so future updates are easier and safer • testing: iterative testing during development and final testing at the end • test plans with normal, boundary, invalid and erroneous data and expected results • finding and fixing syntax and logic errors; trace tables; testing after changes • conclusion: no single measure is enough; layers needed	8 AO1 / AO2 / AO3 2.3.1 / 2.3.2	

Question	Answer	Marks	Guidance
8(b)	**Level 3 (high), 6–8 marks**: Thorough knowledge and understanding of the topic with detailed, accurate and relevant points. A balanced, well-structured answer reaching a justified conclusion where asked. Technical terms used correctly throughout. **Level 2 (mid), 3–5 marks**: Reasonable knowledge and understanding. Some relevant points made, but the answer lacks detail, balance or a clear structure in places. **Level 1 (low), 1–2 marks**: Basic knowledge. A few simple, mostly isolated points with little or no development or linking. **Level 0, 0 marks**: No response or no response worthy of credit. **Indicative content** (credit any other valid points): • testing finds errors before the user does; reduces cost and risk; reduces reputation damage • syntax errors prevent the code running and are found when it is translated; logic errors give wrong results and need careful testing • run-time errors occur while running (e.g. dividing by zero) • iterative testing at each stage means small problems are fixed early • final testing checks the whole program meets the requirements • test data: normal (typical), boundary (edges of range), invalid (outside range), erroneous (wrong type) • test plans record expected vs actual results so that tests can be repeated • limitations: testing cannot prove there are no errors; conclusion that it is essential for reliable software	8 AO1 / AO2 / AO3 2.3.2	

Where marks are lost in Topic 2.3

Use this page after marking. Tick the ones that cost you marks, then redo those questions closed-book.

- ☐ **Validation checks the data is reasonable, not that it is correct.** 'Checks the data is correct' loses the mark.
- ☐ **Five validation types:** range, length, type, presence, format. Be ready to give an example of each.
- ☐ **Four test data types:** normal, boundary (edges, both sides), invalid (wrong value, right type), erroneous (wrong type).
- ☐ **Syntax error** = program won't run; **logic error** = runs but wrong output. Always add the effect.
- ☐ **Maintainability:** comments, meaningful names, indentation, subprograms — name three and say why they help.
- ☐ **Authentication ≠ validation.** Authentication proves who you are; validation checks data entered.
- ☐ **Boundary tests:** if the range is 11–16, test 10, 11, 16 and 17.
- ☐ **Trace tables** find logic errors by following variable values line by line.